Chapter 1 Basics concepts of hydrological models: routing

Olivier Bonte

2026-07-05

To get more insight into the routing concepts explained in the theory, a computational guide is given here, leveraging both symbolic and numerical computing.

```
# Imports
import matplotlib.pyplot as plt
import numpy as np
import sympy as sp
import scipy
import pandas as pd
```

1 Reservoir cascade models

1.1 Cascade of linear reservoirs

1.1.1 First order linear reservoir with constant input

A linear reservoir with storage S [m^3] and a constant input I [m^3/s] is characterised by

$$S = KQ \tag{1}$$

with K [s] the residence time and Q [m^3/s] the outflow. The mass balance equation is given by

$$\frac{dS}{dt} = K\frac{dQ}{dt} = I - Q \tag{2}$$

As this is a linear ordinary differential equation, it can be solved analytically using Laplace transforms (see your course in the 2nd bachelor year on Differential Equations). Remember that although the definition of a Laplace transform might seem complex

$$F(s) = \mathcal{L}\{f(t)\} = \int_0^\infty f(t)e^{-st}dt,$$

in practice you only need to apply a limited set of transforms to solve linear ODEs, which are summarised in the table below.

$f(t)$	$F(s) = \mathcal{L}\{f(t)\}$
$H(t)$	$\frac{1}{s}$
t^n	$\frac{n!}{s^{n+1}}$
e^{at}	$\frac{1}{s-a}$
$f'(t)$	$sF(s) - f(0)$

where $H(t)$ is the Heaviside function, which is defined as $H(t) = 0$ for $t < 0$ and $H(t) = 1$ for $t \geq 0$. n is a positive integer and a is a real number. Applying the Laplace transform to equation Equation 2 with $Q(t = 0) = Q_0 = S_0/K$ and $q(s) = \mathcal{L}\{Q(t)\}$ gives:

$$K(sq(s) - Q_0) = \frac{I}{s} - q(s)$$

Isolating $q(s)$ gives:

$$q(s) = \frac{I}{s(1 + Ks)} + \frac{KQ_0}{1 + Ks} \tag{3}$$

We now need to apply the inverse Laplace transform to get $Q(t)$. Trivially, $\mathcal{L}^{-1}(q(s)) = Q(t)$. For the term containing Q_0:

$$\mathcal{L}^{-1}\left(\frac{KQ_0}{1 + Ks}\right) = \frac{K}{K}Q_0\mathcal{L}^{-1}\left(\frac{1}{1/K + s}\right) = Q_0e^{-t/K} \tag{4}$$

The term with I is a bit more convolved, but can be solved using partial fraction decomposition:

$$\frac{I}{s(1 + Ks)} = \frac{A}{s} + \frac{B}{1 + Ks}$$

To solve for A, multiply both sides by s and set $s = 0$:

$$\frac{I}{1+Ks}\Big|_{s=0} = A \iff A = I$$

To solve for B, multiply both sides by $(1 + Ks)$ and set $s = -1/K$:

$$\frac{I}{s}\Big|_{s=-1/K} = B \iff B = -IK$$

Now we can apply the inverse laplace transform on the partial fraction decomposition:

$$\mathcal{L}^{-1}\left(\frac{I}{s} - \frac{IK}{1+Ks}\right) = \mathcal{L}^{-1}\left(\frac{I}{s} - \frac{I}{s+1/K}\right) = I - Ie^{-t/K} = I(1-e^{-t/K}) \tag{5}$$

Combining Equation 4 and Equation 5 gives the final solution for $Q(t)$:

$$Q(t) = I(1-e^{-t/K}) + Q_0 e^{-t/K} \tag{6}$$

E.g. for $Q_0 = 0.1$ m^3/h and $K = 7$ h, we can plot the outflow $Q(t)$ for different constant inflows I. In the examples that follow, we'll use m^3/h and h for the units instead of the standard SI-units for convenience. Define the analytical solutions as a function:

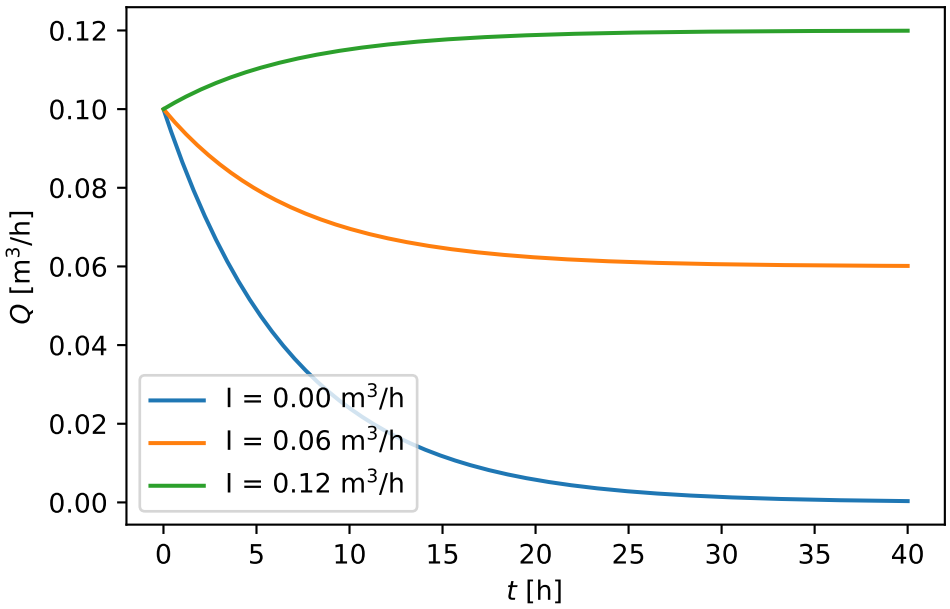
```
def Q_analytical_linear(t, I, K, Q0):
    return I * (1 - np.exp(-t / K)) + Q0 * np.exp(-t / K)
```

define the parameters and the three different inflows:

```
Q_0_num = 0.1  # m^3/h
K_num = 7  # h
t_array = np.linspace(0.01, 40, 200)  # h
I_array = [0.0, 0.06, 0.12]  # m^3/h
```

Note that all numerical examples that follow will use the value of K defined above. Now we can plot the results:

```
fig, ax = plt.subplots()
colors = plt.rcParams["axes.prop_cycle"].by_key()["color"]
for i, I_ in enumerate(I_array):
    Q_ = Q_analytical_linear(t_array, I_, K_num, Q_0_num)
    ax.plot(t_array, Q_, color=colors[i], label=rf"I = {I_:.2f} m$^3$/h")
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.legend()
```



Additionally, note that we can automate the algebraic routines above using SymPy. First define all symbols:

```
K, I, Q0 = sp.symbols("K I Q0", positive=True)
t = sp.symbols("t", positive=True)
s = sp.symbols("s")
q, Q = sp.symbols("q, Q", cls=sp.Function)
```

and then define the ODE in the form $K\frac{dQ}{dt} - (I - Q) = 0$:

```
ode_linear = K * sp.diff(Q(t), t) - (I - Q(t))
ode_linear
```

$$-I + K\frac{d}{dt}Q(t) + Q(t)$$

Now take the Laplace transform of the ODE defining $q(s) = \mathcal{L}\{Q(t)\}$:

```
ode_linear_laplace = sp.laplace_transform(ode_linear, t, s, noconds=True)
ode_linear_laplace = sp.laplace_correspondence(ode_linear_laplace, {Q: q})
ode_linear_laplace
```

$$-\frac{I}{s} + K\left(sq(s) - Q(0)\right) + q(s)$$

We now insert the initial condition $Q(0) = Q_0$:

```
ode_linear_laplace_ic = sp.laplace_initial_conds(ode_linear_laplace, t, {Q:
↪  [Q0]})
ode_linear_laplace_ic
```

$$-\frac{I}{s} + K\left(-Q_0 + sq(s)\right) + q(s)$$

In the equation above, the only unknown is $q(s)$, which can be solved for:

```
q_s_solved = sp.solve(ode_linear_laplace_ic, q(s))[0]
q_s_solved
```

$$\frac{I + KQ_0 s}{s\left(Ks + 1\right)}$$

Now apply the inverse Laplace transform to get $Q(t)$:

```
Q_solved = sp.simplify(sp.inverse_laplace_transform(q_s_solved, s, t))
Q_solved
```

$$\left(Ie^{\frac{t}{K}} - I + Q_0\right)e^{-\frac{t}{K}}$$

Which is identical to the analytical solution Equation 6 derived above.

1.1.2 Second order linear reservoir with constant input

We now get two reservoirs in series:

$$K^{(1)}\frac{dQ^{(1)}}{dt} = I - Q^{(1)}$$
$$K^{(2)}\frac{dQ^{(2)}}{dt} = Q^{(1)} - Q^{(2)}$$

The same SymPy approach can be extended to solve this coupled system. To simplify the solution, we'll assume:

- The second reservoir is initially empty, i.e. $Q^{(2)}(0) = 0$.
- The first reservoir has an initial outflow $Q^{(1)}(0) = Q_0$ as it is assumed non-empty.
- The first and second reservoirs have the same residence time, i.e. $K^{(1)} = K^{(2)} = K$.

We define two unknown functions and their Laplace transforms:

```
q1, q2, Q1, Q2 = sp.symbols("q1 q2 Q1 Q2", cls=sp.Function)
```

Write both ODEs in the form $(\dots) = 0$:

```
ode1 = K * sp.diff(Q1(t), t) - (I - Q1(t))
ode2 = K * sp.diff(Q2(t), t) - (Q1(t) - Q2(t))
display(ode1, ode2)
```

$$-I + K\frac{d}{dt}Q_1(t) + Q_1(t)$$

$$K\frac{d}{dt}Q_2(t) - Q_1(t) + Q_2(t)$$

Take the Laplace transform using $q_1(s) = \mathcal{L}\{Q_1(t)\}$ and $q_2(s) = \mathcal{L}\{Q_2(t)\}$:

```
ode1_laplace = sp.laplace_transform(ode1, t, s, noconds=True)
ode1_laplace = sp.laplace_correspondence(ode1_laplace, {Q1: q1})

ode2_laplace = sp.laplace_transform(ode2, t, s, noconds=True)
ode2_laplace = sp.laplace_correspondence(ode2_laplace, {Q1: q1, Q2: q2})
```

Insert the initial conditions:

```
ode1_laplace_ic = sp.laplace_initial_conds(ode1_laplace, t, {Q1: [Q0], Q2:
↪  [0]})
ode2_laplace_ic = sp.laplace_initial_conds(ode2_laplace, t, {Q1: [Q0], Q2:
↪  [0]})
display(ode1_laplace_ic, ode2_laplace_ic)
```

$$-\frac{I}{s} + K\left(-Q_0 + sq_1(s)\right) + q_1(s)$$

$$Ksq_2(s) - q_1(s) + q_2(s)$$

Solve the algebraic system for $q_1(s)$ and $q_2(s)$:

```
q_s_solved_2 = sp.solve([ode1_laplace_ic, ode2_laplace_ic], [q1(s), q2(s)],
↪  dict=True)[0]
q2_s = q_s_solved_2[q2(s)]
q2_s
```

$$\frac{I + KQ_0 s}{K^2 s^3 + 2Ks^2 + s}$$

To get a solution closer to the notation of the theory, we'll first break up into partial fractions:

```
q2_s_partial = sp.apart(q2_s, s)
q2_s_partial
```

$$-\frac{IK}{Ks+1} + \frac{I}{s} - \frac{K\left(I - Q_0\right)}{\left(Ks+1\right)^2}$$

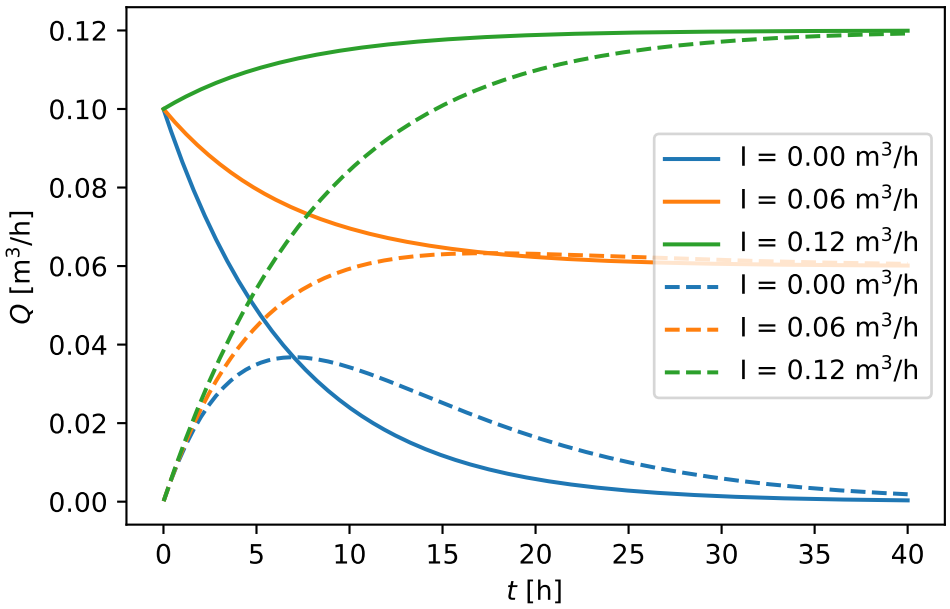
Finally, apply the inverse Laplace transform to obtain $Q_2(t)$.

```
Q2_solved = sp.inverse_laplace_transform(q2_s_partial, s, t)
Q2_solved
```

$$I - Ie^{-\frac{t}{K}} - \frac{t\left(I - Q_0\right)e^{-\frac{t}{K}}}{K}$$

Also $Q_2(t)$ can now be visualised for different constant inflows I and the same parameters as before:

```
for i, I_ in enumerate(I_array):
    Q2_func = sp.lambdify(t, Q2_solved.subs({I: I_, K: K_num, Q0: Q_0_num}),
↪   "numpy")
    Q2_ = Q2_func(t_array)
    ax.plot(
        t_array, Q2_, label=rf"I = {I_:.2f} m$^3$/h", color=colors[i],
↪   linestyle="--"
    )
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.legend()
fig
```



In this figure, full lines are the output of the first reservoir and dashed lines are the output of the second reservoir. Note that the second reservoir has a delayed response compared to the first reservoir.

Given that the second reservoir is initially empty and receives no external input I, Q_2 is entirely defined by Q_1 and $K^{(1)} = K$. So if you solve for Q_2 (in the s-domain $q_2(s)$) with an assumed known $Q_1(t)$ function (in the s-domain $q_1(s)$), you get:

```
q2_s_bis = sp.solve(ode2_laplace_ic, q2(s))[0]
q2_s_bis
```

$$\frac{q_1(s)}{Ks+1}$$

This concept can be generalised in what is called a **transfer function**, which is defined as the ratio of the Laplace transforms of the output and input of a system. In this case, the transfer function $H(s)$ is defined as:

$$H(s) = \frac{q_2(s)}{q_1(s)} = \frac{1}{1+Ks} \tag{7}$$

1.1.3 n-th order linear reservoir with constant input

The concept of 2 reservoirs can be generalised to n reservoirs in series, with the general form of the ODEs given by:

$$\frac{dS^{(i)}}{dt} = K\frac{dQ^{(i)}}{dt} = (Q^{(i-1)} - Q^{(i)}), \quad i = 1, 2, \ldots, n$$

Again, we assume $K^{(i)} = K$ for all reservoirs, $Q^{(0)}(t) = I$ and $Q^{(i)}(t = 0) = 0 \quad \forall i > 1$. The easiest way to solve this equation is to use the transfer function concept. The first reservoir is a special case, as it has an external input I and an initial outflow $Q^{(1)}(0) = Q_0$. In the s-domain, $q^{(1)}(s)$ is given by Equation 3. $\forall i > 1$, the transfer function is given by a form analogous to Equation 7:

$$H^{(i)}(s) = \frac{q_i(s)}{q_{i-1}(s)} = \frac{1}{1 + Ks} \tag{8}$$

Consequently, $q^{(n)}(s)$ can be expressed as:

$$q^{(n)}(s) = q^{(1)}(s)\prod_{i=2}^{n} H^{(i)}(s) = \frac{q^{(1)}(s)}{(1 + Ks)^{n-1}} \tag{9}$$

In SymPy, this can be expressed as (reusing the previously solved $q^{(1)}(s)$ denoted as `q_s_solved`):

```
n = sp.symbols("n", positive=True, integer=True)
q_n_s = q_s_solved / ((1 + K * s) ** (n - 1))
q_n_s = sp.refine(q_n_s, sp.Q.gt(n, 1))
display(q_n_s)
```

$$\frac{(I + KQ_0 s)\,(Ks + 1)^{1-n}}{s\,(Ks + 1)}$$

The inverse Laplace transform can be applied to get $Q^{(n)}(t)$:

```
Q_n_solved = sp.refine(sp.inverse_laplace_transform(q_n_s, s, t), sp.Q.gt(n,
↪ 1))
Q_n_solved
```

$$I\mathcal{L}_s^{-1}\left[\frac{(Ks + 1)^{1-n}}{Ks^2 + s}\right](t) + KQ_0\frac{\partial}{\partial t}\mathcal{L}_s^{-1}\left[\frac{(Ks + 1)^{1-n}}{Ks^2 + s}\right](t)$$

Unfortunately, we run into a limitation of SymPy here, as it can't generalise the inverse Laplace transform for arbitrary n (try calling `sp.simplify` on `Q_n_solved`, it will crash). For specific values of n, we can still get the solutions

```
n_num_list = [5, 6, 7]
for n_num in n_num_list:
    print(f"n = {n_num}")
    Q_n_solved_specific = sp.simplify(Q_n_solved.subs(n, n_num))
    display(Q_n_solved_specific)
```

```
n = 5
```

$$I\left(1 - e^{-\frac{t}{K}} - \frac{te^{-\frac{t}{K}}}{K} - \frac{t^2e^{-\frac{t}{K}}}{2K^2} - \frac{t^3e^{-\frac{t}{K}}}{6K^3} - \frac{t^4e^{-\frac{t}{K}}}{24K^4}\right) + \frac{Q_0t^4e^{-\frac{t}{K}}}{24K^4}$$

```
n = 6
```

$$I\left(1 - e^{-\frac{t}{K}} - \frac{te^{-\frac{t}{K}}}{K} - \frac{t^2e^{-\frac{t}{K}}}{2K^2} - \frac{t^3e^{-\frac{t}{K}}}{6K^3} - \frac{t^4e^{-\frac{t}{K}}}{24K^4} - \frac{t^5e^{-\frac{t}{K}}}{120K^5}\right) + \frac{Q_0t^5e^{-\frac{t}{K}}}{120K^5}$$

```
n = 7
```

$$I\left(1 - e^{-\frac{t}{K}} - \frac{te^{-\frac{t}{K}}}{K} - \frac{t^2e^{-\frac{t}{K}}}{2K^2} - \frac{t^3e^{-\frac{t}{K}}}{6K^3} - \frac{t^4e^{-\frac{t}{K}}}{24K^4} - \frac{t^5e^{-\frac{t}{K}}}{120K^5} - \frac{t^6e^{-\frac{t}{K}}}{720K^6}\right) + \frac{Q_0t^6e^{-\frac{t}{K}}}{720K^6}$$

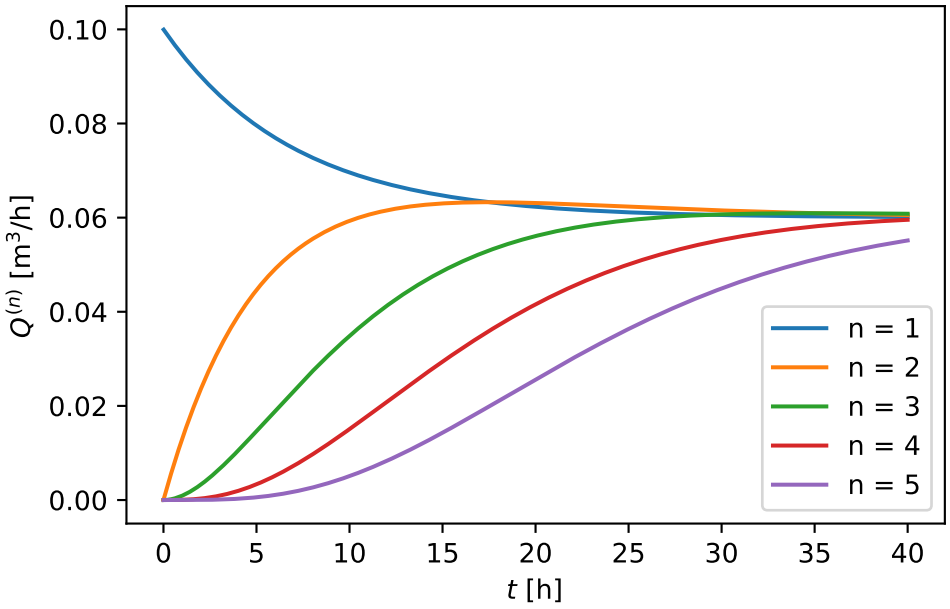
From these solutions, we can easily identify the general pattern that can be expressed as:

$$Q^{(n)}(t) = I - Ie^{-t/K}\sum_{i=0}^{n-1}\frac{(t/K)^i}{i!} + Q_0e^{-t/K}\sum_{i=0}^{n-1}\frac{(t/K)^i}{i!}$$

which is identical to the formulation in the theory notes. We can now also easily plot results for different n values

```
I_num = 0.06  # m^3/h
n_num_list = [1, 2, 3, 4, 5]
fig, ax = plt.subplots()
for n_num in n_num_list:
    Q_n_solved_specific_ = sp.simplify(Q_n_solved.subs(n, n_num))
    Q_n_func_ = sp.lambdify(
        t, Q_n_solved_specific_.subs({I: I_num, K: K_num, Q0: Q_0_num}),
↪   "numpy"
    )
    Q_n_ = Q_n_func_(t_array)
    ax.plot(t_array, Q_n_, label=rf"n = {n_num}")
```

```
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q^{(n)}$ [m$^3$/h]")
ax.legend()
```



The larger n, the more delayed the response of the outflow is.

1.1.4 Nash-casacde

A Nash-cascade is a special case of the n-th order linear reservoir, where the input I to the first reservoir is a Dirac delta function $\delta(t)$:

$$\delta(t) = \begin{cases} \infty, & t = 0 \\ 0, & t \neq 0 \end{cases}$$

with

$$\int_{-\infty}^{\infty} \delta(t)dt = 1$$

It has the very useful property that $\mathcal{L}\{\delta(t)\} = 1$[1]. Additionally assuming that the first reservoir is initially empty, i.e. $Q^{(1)}(0) = 0$, the continuity equation for the first reservoir of Equation 2 sets $I = \delta(t)$ and $Q(t = 0) = 0$. The ODE can be laplace transformed:

[1]Note that t needs to be defined between $-\infty$ and ∞ for this to be true

```
t = sp.symbols("t")  # Generalise t for correct dirac delta transform!
I_dirac = sp.DiracDelta(t)
ode_linear_nash_1 = K * sp.diff(Q(t), t) - (I_dirac - Q(t))
ode_linear_nash_1_laplace = sp.laplace_transform(ode_linear_nash_1, t, s,
 ↪  noconds=True)
ode_linear_nash_1_laplace =
 ↪  sp.laplace_correspondence(ode_linear_nash_1_laplace, {Q: q})
display(ode_linear_nash_1_laplace)
```

$$K\left(sq(s) - Q(0)\right) + q(s) - 1$$

where we can clearly see that the Laplace transform of the Dirac delta function is 1. Inserting $Q(0) = 0$ gives and solving for $q(s)$ gives:

```
ode_linear_nash_1_laplace_ic = sp.laplace_initial_conds(
    ode_linear_nash_1_laplace, t, {Q: [0]}
)
q_1_nash = sp.solve(ode_linear_nash_1_laplace_ic, q(s))[0]
display(q_1_nash)
```

$$\frac{1}{Ks + 1}$$

So in the case of a dirac delta input, also the first reservoir has a transfer function of the same form as the other reservoirs (see Equation 8).

$$H^{(1)}(s) = \frac{q_1(s)}{I(s)} = \frac{q_1(s)}{1} = \frac{1}{1 + Ks}$$

Therefore, Equation 9 can be simplified to:

$$q^{(n)}(s) = \prod_{i=1}^{n} H^{(i)}(s) = \frac{1}{(1 + Ks)^n}$$

Transforming back to the time domain gives:

```
q_n_s_nash = 1 / ((1 + K * s) ** n)
Q_n_solved_nash = sp.simplify(sp.inverse_laplace_transform(q_n_s_nash, s, t))
Q_n_solved_nash
```

$$\frac{K^{-n} t^{n-1} e^{-\frac{t}{K}} \theta\left(t\right)}{\left(n-1\right)!}$$

In the above equation, $\theta(t)$ is the Heaviside function (cf. supra for definition). As we are only interested in $t \geq 0$ (i.e. after the input is applied) we can simplify the solution to:

```
Q_n_solved_nash_simplified = Q_n_solved_nash.subs(
    sp.Heaviside(t), sp.Heaviside(t).rewrite(sp.Piecewise)
)
Q_n_solved_nash_simplified = sp.refine(Q_n_solved_nash_simplified,
 ↪  sp.Q.positive(t))
Q_n_solved_nash_simplified
```

$$\frac{K^{-n} t^{n-1} e^{-\frac{t}{K}}}{\left(n-1\right)!}$$

Again note the equivalence to the formulation in the theory notes. Interestingly, the solution here assumes an empty first reservoir and a Dirac delta input, while the theory notes assume no input and the first reservoir contains a unit volume $S^{(1)}(t=0) = 1\ \mathrm{m}^3$. The two assumptions are equivalent, as the Dirac delta input can be interpreted as this unit input to an empty reservoir.

Because the Nash-cascade is the response to an instantaneous input, it can be interpreted as an Instantaneous Unit Hydrograph (IUH)! Also note that the assumption that n is an integer can be relaxed to n being a positive real number. In this case, applying the inverse Laplace transform yields:

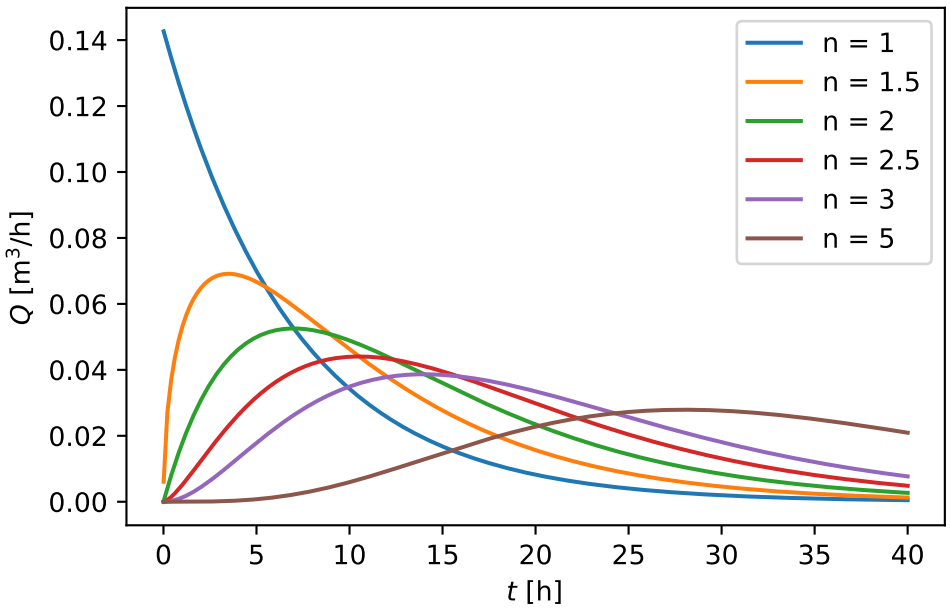
```
n = sp.symbols("n", positive=True, real=True)
q_n_s_nash = 1 / ((1 + K * s) ** n)
Q_n_solved_nash = sp.simplify(sp.inverse_laplace_transform(q_n_s_nash, s, t))
Q_n_solved_nash
```

$$\frac{K^{-n} t^{n-1} e^{-\frac{t}{K}} \theta\left(t\right)}{\Gamma\left(n\right)}$$

This allows n to be calibrated as a real number to match the geomorphological characteristics of a catchment. For completeness, we visualise the Nash-cascade for different n values, including non-integer values:

```
n_num_list = [1, 1.5, 2, 2.5, 3, 5]
fig, ax = plt.subplots()
for n_num in n_num_list:
    Q_n_solved_specific_ = sp.simplify(Q_n_solved_nash.subs(n, n_num))
    Q_n_func_ = sp.lambdify(t, Q_n_solved_specific_.subs({K: K_num}),
 ↪  "numpy")
```

```
    Q_n_ = Q_n_func_(t_array)
    ax.plot(t_array, Q_n_, label=rf"n = {n_num}")
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.tick_params(axis="y")
ax.legend()
```



Note 1: Application: routing via convolution

Defining $u(t) = \frac{1}{(n-1)!K}(\frac{t}{K})^{n-1} \exp(-t/K)$ following the nash cascade, the outflow $Q(t)$ is given by the convolution of $u(t)$ with the input $I(t)$:

$$Q(t) = (u * I)(t) = \int_0^t u(\tau) I(t-\tau) d\tau$$

Note that this convolution concept is only valid because the system is **linear** and hence the **principle of superposition** holds!
Define a toy input $I(t)$:

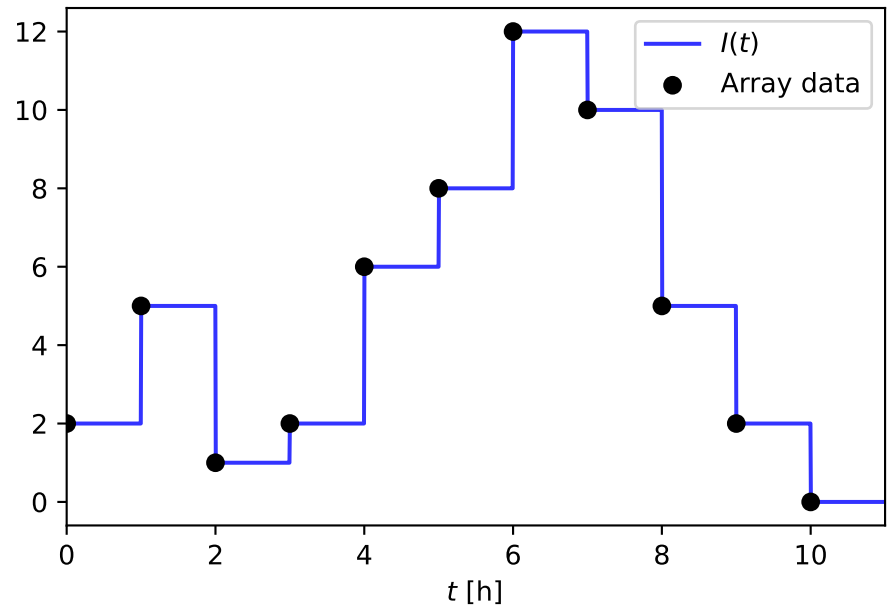
```
I_array = [0.2, 0.5, 0.1, 0.2, 0.6, 0.8, 1.2, 1, 0.5, 0.2, 0.0]  # mm/h
A_example = 10_000  # m^2, example catchment area
I_array_m3h = np.array(I_array) * A_example / 1000  # m^3/h
t_array = np.arange(len(I_array))  # h
```

where each value represent the average rainfall intensity over a 1h time step. We can make a piecewise constant function of this input:

```
I_func = scipy.interpolate.interp1d(
    t_array, I_array_m3h, kind="previous", fill_value=0,
 ↪  bounds_error=False
)
t_eval = np.linspace(0, 60, 61)  # h

# For plotting rainfall take higher resolution timeseries
t_eval_hr = np.linspace(0, 60, 10_000)  # h
I_hr = I_func(t_eval_hr)
```

Where $I(t) = I_m$ during the time interval $[m\Delta t, (m+1)\Delta t[$, as illustrated below.

```
fig, ax = plt.subplots()
ax.plot(t_eval_hr, I_hr, color="blue", label=r"$I(t)$", alpha=0.8)
ax.scatter(t_array, I_array_m3h, color="black", label="Array data",
 ↪  zorder=3)
ax.set_xlim(0, np.max(t_array) * 1.1)
ax.set_xlabel(r"$t$ [h]")
ax.legend()
```



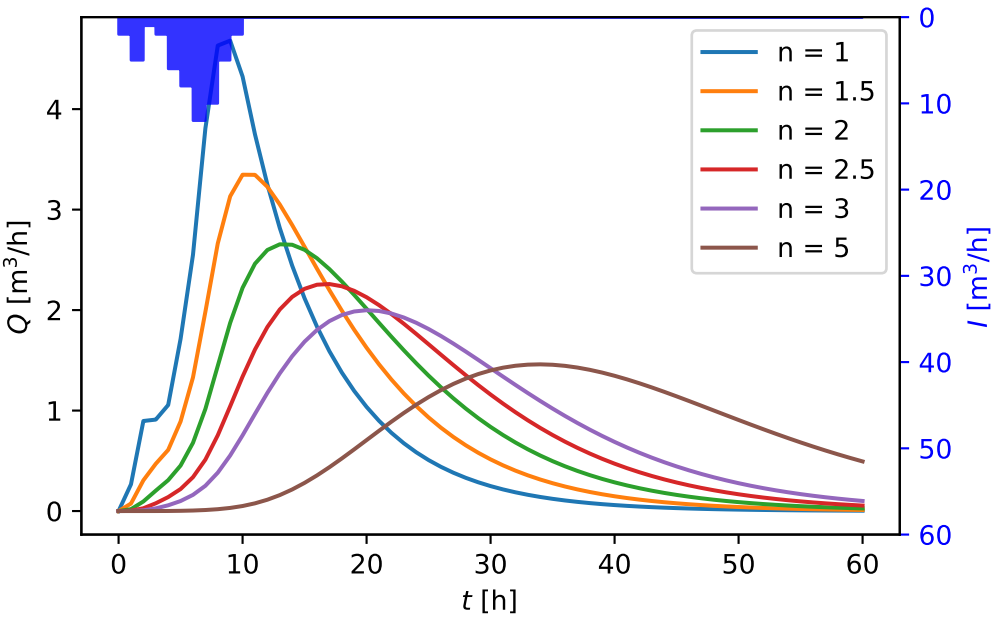
Now wen can numerically integrate the convolution with `scipy.integrate.quad` to get $Q(t)$ for different n values:

```
u_nash_func = sp.lambdify((t, n, K), Q_n_solved_nash, "numpy")


def integrand_func(tau, t, n, K):
    return I_func(tau) * u_nash_func(t - tau, n, K)


Q_numerical_dict = {}
for n_num in n_num_list:
    print("Integrating for n =", n_num)
    Q_numerical_ = np.zeros_like(t_eval)
    for i, t_ in enumerate(t_eval):
        Q_numerical_[i] = scipy.integrate.quad(
            integrand_func, 0, t_, args=(t_, n_num, K_num),
 ↪  points=t_array[t_array <= t_]
        )[0] # Use points argument to handle discontinuities in I_func
    Q_numerical_dict[n_num] = Q_numerical_
```

```
Integrating for n = 1
Integrating for n = 1.5
Integrating for n = 2
Integrating for n = 2.5
Integrating for n = 3
Integrating for n = 5
```

Now we can plot the input and output:

```
fig, ax = plt.subplots()
for n_num, Q_numerical_ in Q_numerical_dict.items():
    ax.plot(t_eval, Q_numerical_, label=rf"n = {n_num}", zorder=3)
ax2 = ax.twinx()
ax2.fill_between(t_eval_hr, I_hr, color="blue", label="Rainfall",
 ↪  alpha=0.8, zorder=2)
ax2.set_ylim(0, np.max(I_hr) * 5)
ax2.set_ylabel(r"$I$ [m$^3$/h]", color="blue")
ax2.tick_params(axis="y", colors="blue")
ax2.invert_yaxis()
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.legend()
```



1.1.5 Linear reservoir with time-varying input

For a linear reservoir with a time-varying input $I(t)$, the ODE is given by:

$$\frac{dS}{dt} = I(t) - \frac{S}{K} \tag{10}$$

For this case, there are 2 options:

1. Apply the convolution approach as shown in Note 1
2. Solve the ODE defined by Equation 10 numerically.

For details on numerical solvers for ODEs, we again refer to your bachelor course on Differential Equations. However, one must pay special attention to the fact the the input $I(t)$ is a piecewise constant function. For example, we consider the rainfall input to be constant over 1h ($= \Delta t_{in}$) time steps:

1. Fixed time step schemes: you choose the time step Δt of your solver to be equal to (or a integer fraction of) the Δt_{in}.
2. Adaptive time step schemes: the solver chooses a timestep Δt based on its estimate of numerical error. These solvers cannot handle discontinuities however. Therefore, you need to integrate in $[0, \Delta t_{in}]$, then in $[\Delta t_{in}, 2\Delta t_{in}]$, etc. and restart the solver at each discontinuity.

These concepts will be illustrated by retaking Note 1 below

Note 2: Routing via numerical ODE integration

Two different numerical solvers (representing the two classes above) will be used to solve the example from Note 1:

1. Explicit Euler method with $\Delta t = \Delta t_{in} = 1$ h
2. The explicit Runge-Kutta method of order 5(4) with adaptive time stepping control as provided by `scipy.integrate.solve_ivp` with the `RK45` method.

We reuse the input function `I(t)` defined in Note 1.

```
def ode_func(t: float, S: np.array, I_func, K: float):
    return I_func(t) - S / K
```

We then implement the explicit Euler method:

```
def explicit_euler(ode_func, t_span:tuple, S0:np.array, delta_t:float,
 ↪  args):
    t_eval = np.arange(t_span[0], t_span[1] + delta_t, delta_t)
    # Initialize the storage array: n_states x n_time_steps
    S = np.zeros((len(S0), len(t_eval)))
    S[:, 0] = S0
    for i in range(1, len(t_eval)):
        dt = t_eval[i] - t_eval[i - 1]
        S[:, i] = S[:, i - 1] + dt * ode_func(t_eval[i - 1], S[:, i - 1],
 ↪  *args)
    return S
```

We can now solve the ODE with this method.

```
S0 = np.array([0]) # Initial storage
t_span = (0, 60) # hr
delta_t = 1 # hr
S_explicit_euler_solution = explicit_euler(ode_func, t_span, S0, delta_t,
 ↪  (I_func, K_num))
```

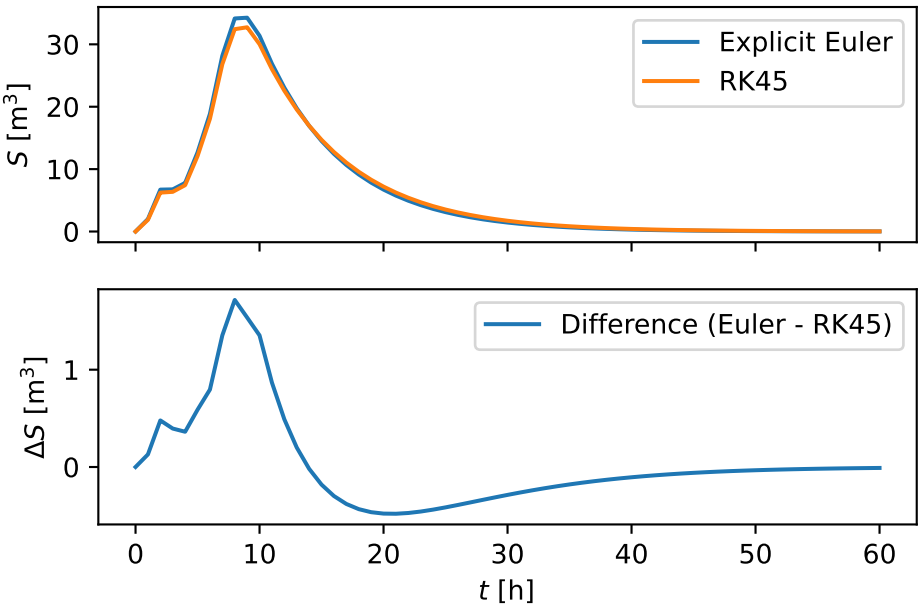
For the RK45 method we write a wrapper around `scipy.integrate.solve_ivp` to handle the piecewise constant input function.

```
def rk_45(ode_func, t_span: tuple, S0: np.array, delta_t_eval: float,
 ↪  args, **kwargs):
    t_eval = np.arange(t_span[0], t_span[1] + delta_t_eval, delta_t_eval)
    S = np.zeros((len(S0), len(t_eval)))
    S[:, 0] = S0  # store initial condition
    S_current = S0
    for i in range(len(t_eval) - 1):
        t_interval = (t_eval[i], t_eval[i + 1])
        sol = scipy.integrate.solve_ivp(
            ode_func,
            t_interval,
            S_current,
            method="RK45",
            args=args,
            t_eval=list(t_interval),
            **kwargs,
        )
        S[:, i + 1] = sol.y[:, -1]
        S_current = sol.y[:, -1]
    return S
```

```
S_rk45_solution = rk_45(ode_func, t_span, S0, delta_t, (I_func, K_num))
```

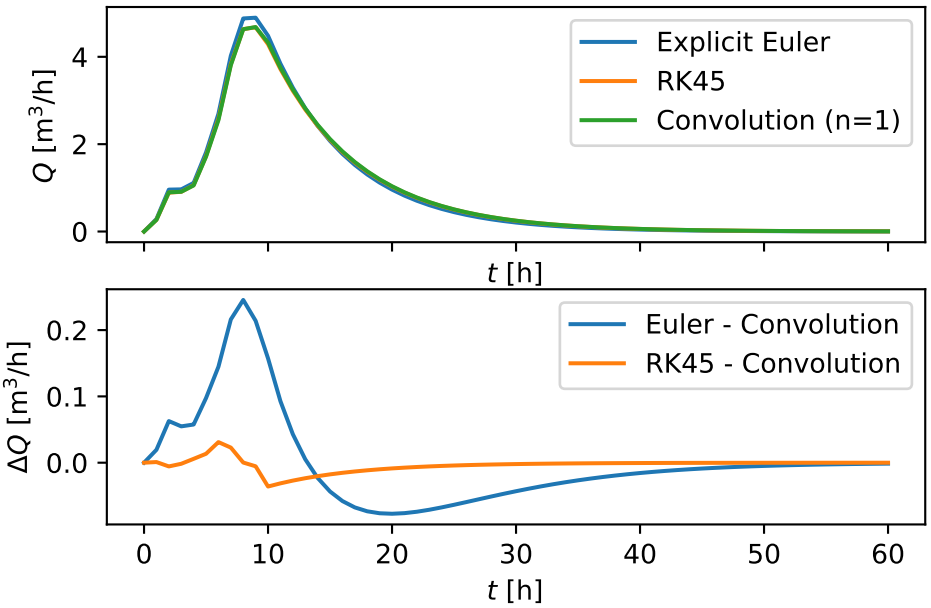
Now we can plot the results of both solvers:

```
diff_S = S_explicit_euler_solution - S_rk45_solution
fig, axes = plt.subplots(2, 1, sharex=True)
axes[0].plot(t_eval, S_explicit_euler_solution[0, :], label="Explicit
 ↪  Euler")
axes[0].plot(t_eval, S_rk45_solution[0, :], label="RK45")
axes[0].set_ylabel(r"$S$ [m$^3$]")
axes[0].legend()

axes[1].plot(t_eval, diff_S[0, :], label="Difference (Euler - RK45)")
axes[1].set_xlabel(r"$t$ [h]")
axes[1].set_ylabel(r"$\Delta S$ [m$^3$]")
axes[1].legend()
```



As $Q = S/K$, this difference directly translates to the outflow $Q(t)$. Below, we compare this Q to the convolution solution from Note 1 for $n = 1$

```
fig, axes = plt.subplots(2, 1, sharex=True)
Q_euler = S_explicit_euler_solution[0, :] / K_num
Q_rk45 = S_rk45_solution[0, :] / K_num
Q_convolution = Q_numerical_dict[1]
axes[0].plot(t_eval, Q_euler, label="Explicit Euler")
axes[0].plot(t_eval, Q_rk45, label="RK45")
axes[0].plot(t_eval, Q_convolution, label="Convolution (n=1)")
axes[0].set_xlabel(r"$t$ [h]")
axes[0].set_ylabel(r"$Q$ [m$^3$/h]")
axes[0].legend()

diff_euler_convolution = Q_euler - Q_convolution
diff_rk45_convolution = Q_rk45 - Q_convolution
axes[1].plot(t_eval, diff_euler_convolution, label="Euler - Convolution")
axes[1].plot(t_eval, diff_rk45_convolution, label="RK45 - Convolution")
axes[1].set_xlabel(r"$t$ [h]")
axes[1].set_ylabel(r"$\Delta Q$ [m$^3$/h]")
axes[1].legend()
```



So these are not identical, although `RK45` is closer to the convolution solution than the explicit Euler method. This is to be expected, as both `RK45` and `scipy.integrate.quad` have numerical error control, while the explicit Euler method does not.
Lastly, we can also do numerical ODE integration for an n-th order linear reservoir with a time-varying input $I(t)$.

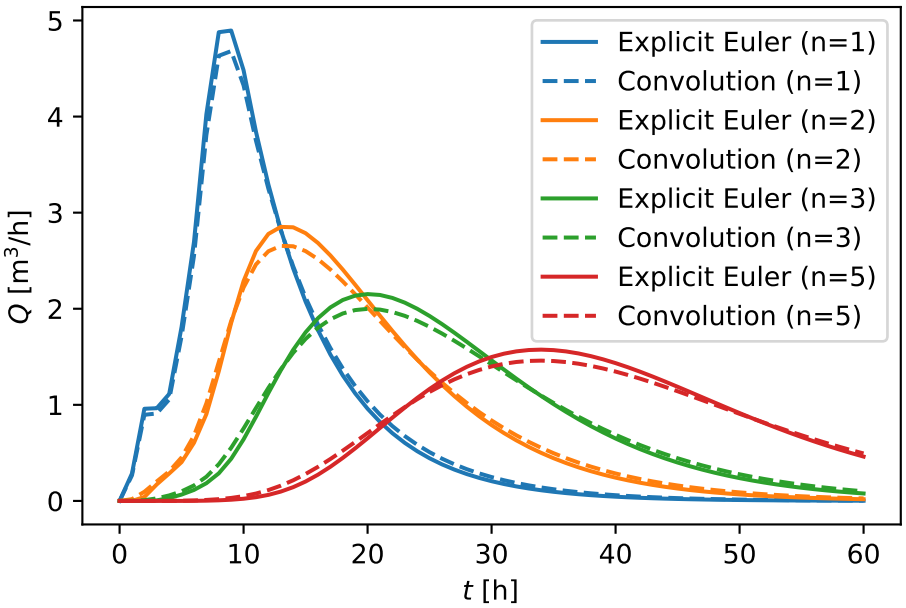
```
def ode_func_nth_order(t: float, S: np.array, I_func, K: float, n: int):
    dSdt = np.zeros_like(S)
    dSdt[0] = I_func(t) - S[0] / K
    if n > 1:
        for i in range(1, n):
            dSdt[i] = S[i - 1] / K - S[i] / K
    return dSdt
```

This can be solved with both solvers. $S^{(i)} = 0$ for all i.

```
S_n_EE_dict = {}
S_n_RK45_dict = {}
n_num_list = [1, 2, 3, 5]
for n_num in n_num_list:
    S0 = np.zeros(n_num)  # Initial storage
    S_n_EE_dict[n_num] = explicit_euler(
        ode_func_nth_order, t_span, S0, delta_t, (I_func, K_num, n_num)
    )
    S_n_RK45_dict[n_num] = rk_45(
        ode_func_nth_order, t_span, S0, delta_t, (I_func, K_num, n_num)
    )
```

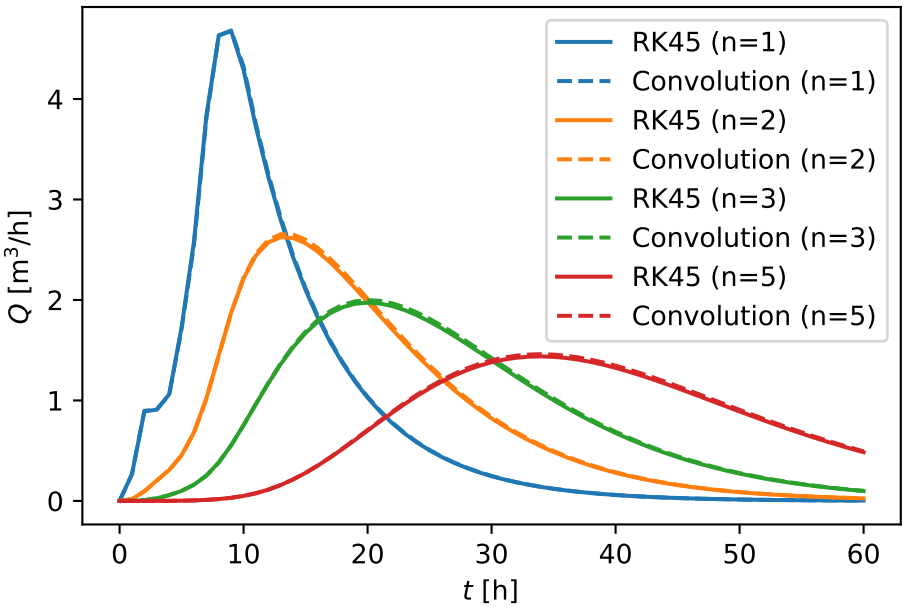
Comparing Explicit Euler to the convolution:

```
fig, ax = plt.subplots()
colors = plt.get_cmap("tab10").colors
for i, n_num in enumerate(n_num_list):
    Q_n_EE = S_n_EE_dict[n_num][n_num - 1, :] / K_num
    ax.plot(t_eval, Q_n_EE, label=f"Explicit Euler (n={n_num})",
 ↪  color=colors[i])
    Q_n_convolution = Q_numerical_dict[n_num]
    ax.plot(
        t_eval,
        Q_n_convolution,
        label=f"Convolution (n={n_num})",
        linestyle="--",
        color=colors[i],
    )
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.legend()
```



and `RK45` to the convolution:

```
fig, ax = plt.subplots()
for i, n_num in enumerate(n_num_list):
    Q_n_RK45 = S_n_RK45_dict[n_num][n_num - 1, :] / K_num
    ax.plot(t_eval, Q_n_RK45, label=f"RK45 (n={n_num})", color=colors[i])
    Q_n_convolution = Q_numerical_dict[n_num]
    ax.plot(
        t_eval,
        Q_n_convolution,
        label=f"Convolution (n={n_num})",
        linestyle="--",
        color=colors[i],
    )
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.legend()
```



The same conclusions as before can be drawn: the RK45 method is closer to the convolution solution than the explicit Euler method. The most important takeaway is that both the convolution approach and the numerical ODE integration can be applied to linear systems with time-varying inputs and yield pratically identical results when numerical error in the solvers are controlled.

1.2 Cascade of non-linear reservoirs

In contrast to Equation 1, the storage-outflow relationship is now given by a non-linear function:

$$Q = CS^r \tag{11}$$

where C [$1/(\mathrm{m}^{3^{r-1}}\ \mathrm{s})$] and r [-] are parameters that can be calibrated. This relation makes the ODE non-linear, and so general analytical solutions cannot be obtained via the Laplace transformation.

The mass balance equation is then given by:

$$\frac{dS}{dt} = I(t) - CS^r$$

This can be solved numerically with the same solvers as before, as shown below.

Note 3: Routing via numerical ODE integration for non-linear reservoirs

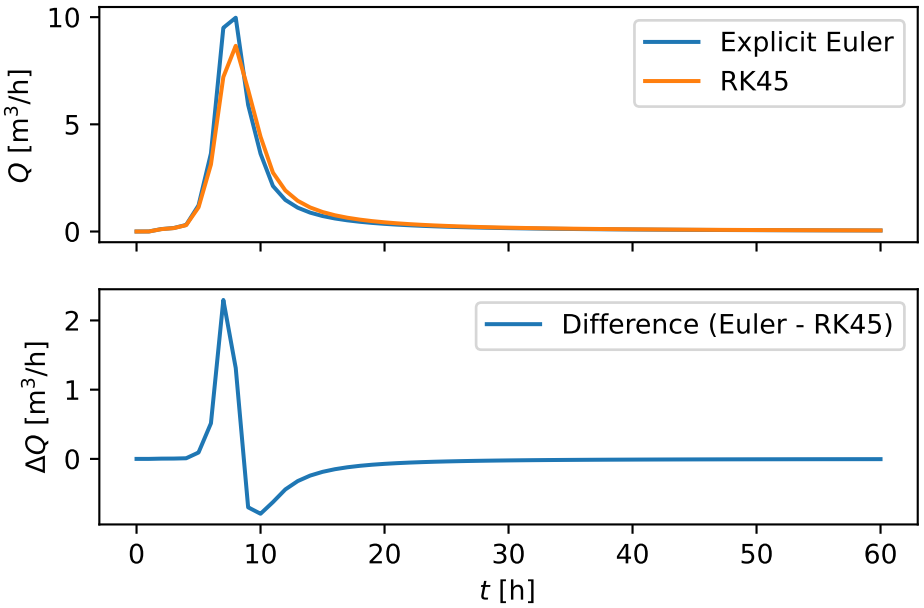
We retake Note 2 but now the storage-outflow relation takes the form of Equation 11 with $r = 3$.

```
def ode_func_nonlinear(t: float, S: np.array, I_func, C: float, r:
 ↪  float):
    dSdt = I_func(t) - C * S**r
    return dSdt
```

The ODE are integrated with both solvers (explicit Euler and `RK45`). Note that we can't use the K residence time parameter anymore, as we can't relate K to C without taking into account the storage itself ($Q = S/K = CS^r$)

```
r_num = 3
C_num = 1 / 3000 # 1/(m^6 h)
S0 = np.array([0])  # Initial storage
sol_ee_nonlinear = explicit_euler(
    ode_func_nonlinear, t_span, S0, delta_t, (I_func, C_num, r_num)
)
Q_ee_nonlinear = (C_num * sol_ee_nonlinear**r_num).flatten()
sol_rk45_nonlinear = rk_45(
    ode_func_nonlinear, t_span, S0, delta_t, (I_func, C_num, r_num)
)
Q_rk45_nonlinear = (C_num * sol_rk45_nonlinear**r_num).flatten()
```

```
diff_euler_rk45_nonlinear = Q_ee_nonlinear - Q_rk45_nonlinear

fig, axes = plt.subplots(2, 1, sharex=True)
axes[0].plot(t_eval, Q_ee_nonlinear, label="Explicit Euler")
axes[0].plot(t_eval, Q_rk45_nonlinear, label="RK45")
axes[0].set_ylabel(r"$Q$ [m$^3$/h]")
axes[0].legend()

axes[1].plot(t_eval, diff_euler_rk45_nonlinear, label="Difference (Euler
 ↪  - RK45)")
axes[1].set_xlabel(r"$t$ [h]")
axes[1].set_ylabel(r"$\Delta Q$ [m$^3$/h]")
axes[1].legend()
```



Remark that the difference between the two solvers is larger than for the linear reservoir case in Note 2, i.e. the numerical error for the explicit Euler scheme becomes substantial. Nonetheless, a lot of operatoinal rainfall-runoff models (which contain non-linear reservoirs) still use the explicit Euler method. Following the seminal work of Clark and Kavetski (n.d.), we encourage the reader to use adaptive explicit (such as `RK45`) or implicit methods.

1.3 Muskingum method

For this method, the storage-outflow relationship is given by:

$$S = KQ + KX(I - Q) = K[XI + (1 - X)Q]$$

This can be rearanged to give the outflow Q as a function of the storage S and the inflow I:

$$Q = \frac{S/K - XI}{1 - X}$$

and so the mass balance equation is given by:

$$\frac{dS}{dt} = I - \frac{S/K - XI}{1 - X} = \frac{I(1 + X)}{1 - X} - \frac{S/K}{1 - X}$$

where $I = I(t)$ as the input is time-varying. Note that the relation between storage and discharge is linear. The Muskingum method is typically applied over a channel reach, defined

by its length Δx. Below, an example is worked out based on Brutsaert (2005). However, the example is 'modernised' with following changes:

1. The explicit Euler method is replaced by the adaptive `RK45` method
2. Calibration will be carried out using standard calibration practices (minimising the mean squared error of the ODE solution) instead of the methods described in Brutsaert (2005).
3. Replace the upper bound on the allowed Δt given the wave celerity $c_m = \Delta x/K$ and X from Brutsaert (2005) for the specific time discretisation proposed there with a more general Courant-Friedrichs-Lewy (CFL) condition: $\frac{c_m \Delta t}{\Delta x} \leq 1 - X$.

Note 4: Example: Muskingum channel routing

Following Brutsaert (2005), $\Delta x = 23$ km. Example data for I and O is given in `data/brutsaert_reach.csv` with `I` and `O` in m^3/s and `t` in h.

```
df_brutsaert = pd.read_csv("data/brutsaert_reach.csv", index_col=0)
df_brutsaert.head()
t_array = df_brutsaert.index.values * 3600  # Convert h to s
delta_x = 23_000  # m
```

For the interpolation, we'll assume that the hourly measurement is reflective of the flow at that exact moment in the time. For simplicity, we'll therefore apply linear interpolation between the measurements

```
fn_I = scipy.interpolate.interp1d(
    t_array, df_brutsaert["I"], kind="linear", fill_value="extrapolate"
)
```

Now define the ODE function for the Muskingum method:

```python
def muskingum_outflow(S, K, X, I):
    return (S / K - X * I) / (1 - X)


def muskingum_storage(Q, K, X, I):
    return K * (X * I + (1 - X) * Q)


def calc_max_delta_t(K, X, delta_x):
    c_m = delta_x / K  # wave celerity
    return (1 - X) * delta_x / c_m  # CFL condition


def ode_func_muskingum(t: float, S: np.array, I_func, K: float, X:
↪   float):
    I_t = I_func(t)
    dSdt = I_t - muskingum_outflow(S, K, X, I_t)
    return dSdt
```

To obtain optimal parameters for K and X, we define a cost function that calculates the mean squared error between the observed outflow and the modelled outflow.

```
def cost_function_muskingum(params, t_array, I_func, O_obs, delta_x):
    K_hours, X = params
    K = K_hours * 3600  # Convert to seconds for the ODE
    # Equilibrium initial storage: S0 = K*[(1-X)*O0 + X*I0]
    S0 = np.array([
        muskingum_storage(O_obs[0], K, X, I_func(t_array[0]))
    ])  # Initial storage
    max_delta_t = calc_max_delta_t(K, X, delta_x)  # CFL condition
    delta_t_eval = np.diff(t_array)[
        0
    ]  # Use the time step of the input data for evaluation
    S_solution = rk_45(
        ode_func_muskingum,
        (t_array[0], t_array[-1]),
        S0,
        delta_t_eval=delta_t_eval,
        args=(I_func, K, X),
        max_step=max_delta_t,
    )  # Solve ODE
    Q_solution = muskingum_outflow(
        S_solution[0, :], K, X, I_func(t_array)
    )  # Calculate outflow
    mse = np.mean((Q_solution - O_obs) ** 2)
    return mse
```

To calibrathe parameters with the Nelder-Mead method from `scipy.optimize.minimize`, we need to provide an initial guess for K and X:

- For the physical interpretation of X tol hold (see Brutsaert (2005)), $X \in]0, 0.5]$ is required. 0.25 is hence a reasonable initial guess.
- K can still be intepreted as a residence time, so $K = 5h$ is assumed a reasonable initial guess.

```
method = "Nelder-Mead"
K_hours_0 = 5.0  # hours (optimizer works in hours; cost function
 ↪  converts to seconds)
X_0 = 0.25  # -
params_0 = np.array([K_hours_0, X_0])
bounds = [(1e-6, None), (1e-6, 0.5)]  # K_hours > 0, X in ]0, 0.5]
test_cost = cost_function_muskingum(params_0, t_array, fn_I,
 ↪  df_brutsaert["O"].values, delta_x)
```

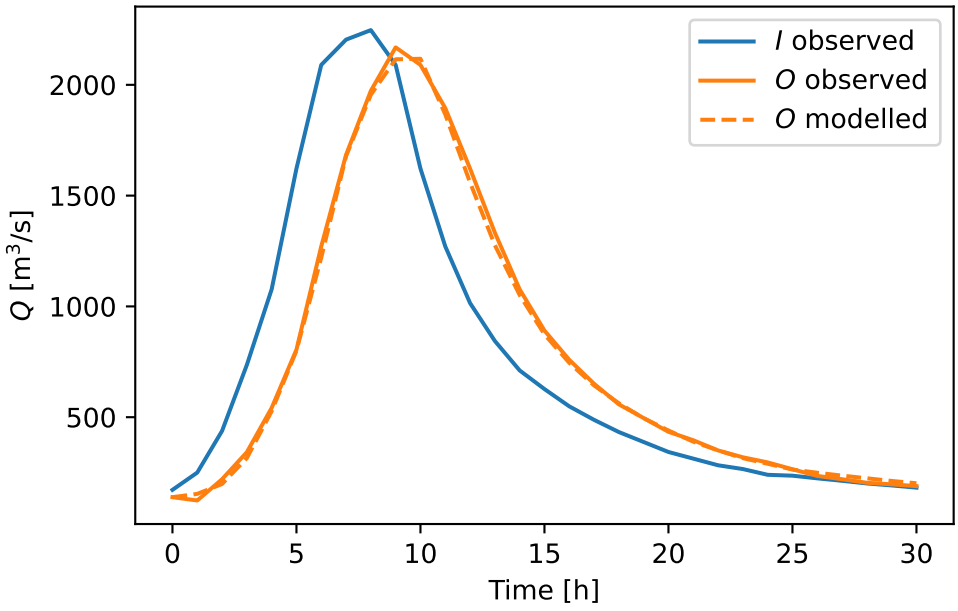
```
optim_res = scipy.optimize.minimize(
    cost_function_muskingum,
    params_0,
    args=(t_array, fn_I, df_brutsaert["O"].values, delta_x),
    method=method,
    bounds=bounds,
    tol=1e-3,
    options={"disp": True},
)
K_hours_opt, X_opt = optim_res.x
K_opt = K_hours_opt * 3600  # Convert to seconds
print(f"Optimized parameters: K = {K_hours_opt:.2f} h, X = {X_opt:.4f}")
```

```
Optimization terminated successfully.
         Current function value: 626.365174
         Iterations: 35
         Function evaluations: 69
Optimized parameters: K = 1.95 h, X = 0.3211
```

These values are very similar to the $K = 1.99$h and $X = 0.3$ reported in Brutsaert (2005). We can now solve the ODE with the optimized parameters and compare the results to the observed outflow:

```
S0_opt = np.array([
    muskingum_storage(df_brutsaert["O"].iloc[0], K_opt, X_opt,
 ↪  fn_I(t_array[0]))
])  # Initial storage
max_step = calc_max_delta_t(K_opt, X_opt, delta_x)  # CFL condition
S_solution_opt = rk_45(
    ode_func_muskingum,
    (t_array[0], t_array[-1]),
    S0_opt,
    delta_t_eval=3600,
    args=(fn_I, K_opt, X_opt),
    max_step=max_step,
)  # Solve ODE
Q_solution_opt = muskingum_outflow(
    S_solution_opt[0, :], K_opt, X_opt, fn_I(t_array)
)  # Calculate outflow
```

```
fig, ax = plt.subplots()
df_brutsaert["I"].plot(ax=ax, label=r"$I$ observed", color=colors[0])
df_brutsaert["O"].plot(ax=ax, label=r"$O$ observed", color=colors[1])
ax.plot(
    t_array / 3600,
    Q_solution_opt,
    label=r"$O$ modelled",
    linestyle="--",
    color=colors[1],
)
ax.set_xlabel("Time [h]")
ax.set_ylabel(r"$Q$ [m$^3$/s]")
ax.legend()
```



References

Brutsaert, Wilfried. 2005. *Hydrology: An Introduction*. Cambridge University Press. https://doi.org/10.1017/CBO9780511808470.

Clark, Martyn P., and Dmitri Kavetski. n.d. "Ancient Numerical Daemons of Conceptual Hydrological Modeling: 1. Fidelity and Efficiency of Time Stepping Schemes." *Water*

Resources Research 46 (10). https://doi.org/10.1029/2009WR008894.